

Day 2 Report: From Counting to Predicting – TF-IDF to Distributional Embeddings

Submitted By:

Manahil

Abstract

This project explores the transition from frequency-based text representation to predictive word embeddings in NLP. TF-IDF was implemented from scratch and verified using Scikit-learn. A cosine similarity retrieval system was developed to evaluate document similarity. Word2Vec Skip-gram was then trained to learn semantic word representations. The results show that TF-IDF cannot capture semantic similarity between synonyms, while Word2Vec performs better but fails to distinguish multiple meanings of the same word. These findings demonstrate the need for contextual embeddings used in modern language models such as GPT.

1. Introduction

Traditional NLP methods such as Bag-of-Words (BoW) and TF-IDF represent text using word frequencies. While effective for information retrieval, they cannot understand semantic relationships between words. Word2Vec introduced predictive learning by generating word embeddings based on surrounding context, allowing semantically related words to have similar vector representations. This project compares TF-IDF and Word2Vec to understand their strengths, limitations, and their connection to modern Large Language Models (LLMs).

2. Objectives

- Implement TF, IDF, and TF-IDF from scratch.
- Verify results using Scikit-learn.
- Build a cosine similarity retrieval system.
- Compare TF-IDF and Word2Vec similarity scores.
- Demonstrate Word2Vec's limitation with polysemous words.
- Relate Word2Vec's predictive learning to modern GPT-style models.

3. Background

Bag-of-Words (BoW): Represents text using word frequencies but ignores context and meaning.

TF-IDF: Assigns higher weights to important words while reducing the impact of common words.

$$TF-IDF = TF \times IDF$$

Word2Vec: Learns word embeddings by predicting surrounding words (Skip-gram), enabling it to capture semantic similarity. However, it assigns one fixed vector to each word regardless of context.

4. Methodology

The project was completed in the following steps:

1. Preprocessed the corpus by converting text to lowercase and tokenizing it into sentences and words.
2. Built a Bag-of-Words (BoW) representation from scratch.
3. Computed **Term Frequency (TF)**, **Inverse Document Frequency (IDF)**, and **TF-IDF** manually.
4. Verified the manually computed TF-IDF values using Scikit-learn's TfidfVectorizer.
5. Developed a cosine similarity retrieval system to rank documents based on a user query.
6. Compared TF-IDF similarity scores for selected synonym pairs.
7. Trained a **Word2Vec Skip-gram** model using Gensim.
8. Evaluated the model using nearest-neighbor queries, word similarities, and analogy tests.
9. Demonstrated the limitation of Word2Vec by showing that a polysemous word receives the same vector regardless of its meaning.

5. Implementation

5.1 Bag-of-Words (BoW)

A custom Bag-of-Words implementation was developed without using Scikit-learn. The corpus was tokenized, a vocabulary of unique words was created, and each sentence was represented as a frequency vector based on the occurrence of vocabulary words.

Outcome:

The BoW model successfully converted textual data into numerical vectors but did not preserve word order or semantic meaning.

5.2 TF-IDF Implementation

TF, IDF, and TF-IDF were implemented from scratch using Python. TF measured the importance of a word within a document, while IDF reduced the weight of words appearing frequently across the corpus. Their product generated the final TF-IDF score for each word.

The implementation was verified using Scikit-learn's TfidfVectorizer, and both approaches produced consistent results.

Outcome:

The manually computed TF-IDF values matched the library implementation, confirming the correctness of the implementation.

5.3 Cosine Similarity Retrieval

A cosine similarity retrieval system was implemented using TF-IDF vectors. The system converts both the query and documents into TF-IDF vectors and calculates the cosine similarity between them. Documents are then ranked based on their similarity scores, with higher scores indicating greater relevance.

Outcome:

The retrieval system successfully ranked documents according to their relevance to the input query.

5.4 Synonym Similarity using TF-IDF

Five synonym/paraphrase pairs were analyzed using TF-IDF similarity. Since TF-IDF represents each word as an independent feature, synonym pairs with different spellings produced very low similarity scores despite having similar meanings.

Outcome:

The experiment demonstrated that TF-IDF relies on exact word matching and cannot capture semantic relationships between different words.

5.5 Word2Vec Skip-gram Training

A Word2Vec Skip-gram model was trained using the Gensim library. The model learns word embeddings by predicting surrounding context words from a given center word. Unlike TF-IDF, this predictive approach enables semantically related words to obtain similar vector representations.

Training Parameters

Parameter	Value
-----------	-------

Vector Size	100
-------------	-----

Window Size	5
-------------	---

Min Count	1
-----------	---

Architecture	Skip-gram (sg = 1)
--------------	--------------------

Epochs	50
--------	----

Outcome:

The model successfully learned distributed word embeddings from the training corpus.

5.6 Model Evaluation

The trained Word2Vec model was evaluated using nearest-neighbor queries, word similarity, and analogy tests. Words appearing in similar contexts were assigned similar vector representations, resulting in higher similarity scores than those obtained using TF-IDF.

Outcome:

Word2Vec captured semantic relationships more effectively than TF-IDF, demonstrating the advantage of predictive learning over frequency-based representations.

5.7 Polysemy Analysis

A polysemous word ("bank") was selected to examine whether Word2Vec generates different vectors for different meanings. The model returned the same embedding regardless of context because it stores only one vector for each word.

Outcome:

The experiment showed that Word2Vec cannot distinguish between multiple meanings of the same word, highlighting the limitation of static word embeddings.

Polysemy Limitation:

The Day 1 corpus did not contain a suitable polysemous word (such as *bank*, *bat*, or *spring*) used in two different senses. Therefore, this limitation could not be demonstrated experimentally. However, Word2Vec assigns exactly one embedding to each word in its vocabulary. If a polysemous word appeared with multiple meanings, the model would still return the same vector because Word2Vec is a static embedding model and does not use surrounding context to create different representations.

6. Results and Discussion

The experiments demonstrated a clear progression from frequency-based methods to predictive embeddings. TF-IDF effectively identified important words and supported document retrieval but failed to recognize semantic similarity between synonyms. In contrast, Word2Vec learned meaningful semantic relationships by considering word context during training, producing better similarity results. However, because Word2Vec generates a single fixed vector for each word, it cannot handle context-dependent meanings. This limitation motivated the development of contextual embedding models such as BERT and GPT.

7. Conclusion

This project successfully implemented TF-IDF from scratch, verified its correctness using Scikit-learn, and developed a cosine similarity retrieval system. A Word2Vec Skip-gram model was then trained to learn distributed word representations. Experimental results showed that Word2Vec outperformed TF-IDF in capturing semantic similarity but was unable to resolve polysemy due to its static embeddings. These findings illustrate the evolution of NLP from counting-based methods to predictive embeddings and emphasize the need for contextual representations, which are now used in modern Transformer-based language models.

8. Key Learnings

- Implemented TF, IDF, and TF-IDF from scratch.
- Verified manual TF-IDF calculations using Scikit-learn.
- Built a cosine similarity document retrieval system.
- Observed TF-IDF's inability to recognize synonyms.
- Trained a Word2Vec Skip-gram model using Gensim.
- Compared TF-IDF and Word2Vec similarity performance.
- Demonstrated the limitation of static embeddings for polysemous words.

- Understood how Word2Vec's predictive training influenced modern GPT-style language models.